

Prime Throughput Performance Evaluation

Report 3

Source Code and GitHub Repositories

The files used for this evaluation can be found at the following repository:

<https://github.com/sam444555/cs2910>

The original Prime code used is available at the following Johns Hopkins DSN repository:

<https://github.com/jhu-dsn/prime>

Revised Latency Measurement Methodology

The approach to latency measurement has been modified since the last report. Previously, the timer used to measure the latency of an arbitrary update n did not start until the update had been successfully sent in the `Send_Update(...)` function. This approach failed to accurately capture the back pressure experienced by some updates while waiting to be sent to Prime daemon, resulting in a misrepresentation of actual end-to-end latency.

Under the revised approach, once a sending stall is encountered, the update's latency timer will begin. Room exists in the network pipeline ($num_outstanding_updates < num_clients_to_emulate$); therefore, the update should be inserted and its latency timer started regardless of whether it can be sent immediately to the Prime daemon. This incorporates the back pressure-induced delay into the latency measurement, resulting in a more accurate latency representation experienced by each update. This applies to all updates in the current function invocation ($n, n+1, n+2, \dots$) until the entire network pipeline becomes full or no updates remain to be sent.

Send_Update function modifications

The `Send_Update(...)` function was modified to support the revised approach. The primary change is that the main loop continues whenever a stall is encountered instead of returning immediately. Once detected, $num_outstanding_updates$ and $curr_seq_num$ at the current iteration are recorded for rollback purposes & every subsequent send starts the latency timer for the current sequence number without creating and sending an actual update.

Once the loop exits, the rollback is applied and undoes the changes made to the aforementioned variables so follow up function invocations can correctly attempt to resend the stalled updates. A socket event handler was also added to monitor availability and send a subsequent `Send_Update(...)` once the socket became writable. This handler gets added outside the main loop body only if a sending stall was detected (indicated by $stall_detected$ flag) otherwise it is detached to prevent unnecessary calls.

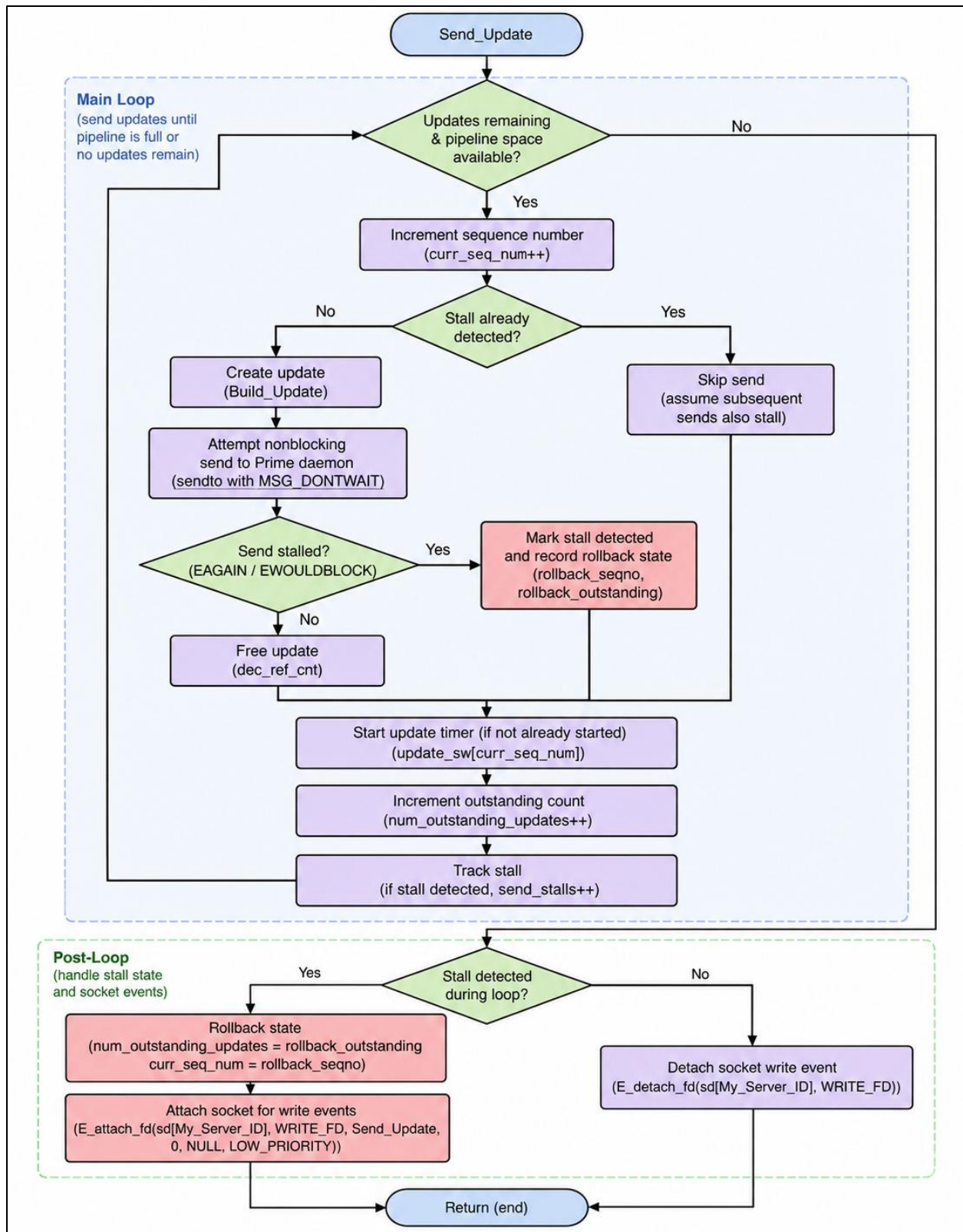


Figure 1. `Send_Update` function logic flowchart

~~Sending Freeze Resolved~~

~~These changes resolved the occasional freezing behavior that had previously affected the driver program. I have since been unable to reproduce this issue even under the most demanding test conditions (100,000 total updates and 150 emulated clients).~~

The root of the client-side sending issues can be traced back to Prime itself, specifically the **MAX_PO_IN_FLIGHT** configuration parameter located in *prime/src/def.h*. Once **MAX_PO_IN_FLIGHT** has been reached or exceeded, Prime detaches the client socket from `READ_FD` (*PRE_ORDER_Send_PO_Request(...)* located in *prime/src/pre_order.c*), temporarily preventing additional client requests until more capacity becomes available. It reattaches in *prime/src/order.c* when the difference between the most recently created PO sequence and the most recently executed PO sequence is exactly equal to **MAX_PO_IN_FLIGHT**.

I suspected the issue was Prime failing to reattach the client socket. To investigate this theory, I ran 25 driver tests using the following input:

- **Number of emulated clients = 50**
- **Total Updates = 20k**
- **MAX_PO_IN_FLIGHT = 20** (default value)
- **No bandwidth limitation**
- **Single driver running on prime1**

```
RESPONSE: seq=19996 | curr_seq=20000 | outstanding=4 | time=19.161555 sec
RESPONSE: seq=19997 | curr_seq=20000 | outstanding=3 | time=19.161557 sec
RESPONSE: seq=19998 | curr_seq=20000 | outstanding=2 | time=19.161558 sec
RESPONSE: seq=19999 | curr_seq=20000 | outstanding=1 | time=19.161560 sec
```

I continuously observed that the program would freeze when there was 1 outstanding update remaining (19,999/20,000 successfully processed). I encountered the same failure conditions again, this time with the Prime daemon logging its socket attach and detach operations.

```
prime1 | 10/02/26 09:45:54.941945 DETACH: po_seq=171362 | executed=171342 | diff=20
prime1 | 10/02/26 09:45:54.960190 REATTACH: po_seq=171362 | executed=171342 | diff=20
```

☒ View in Docker Desktop ☐ View Config ☐ Enable Watch ☐ Detach

```
RESPONSE: seq=19997 | curr_seq=20000 | outstanding=3 | time=19.245185 sec
RESPONSE: seq=19998 | curr_seq=20000 | outstanding=2 | time=19.245186 sec
RESPONSE: seq=19999 | curr_seq=20000 | outstanding=1 | time=19.245196 sec
```

Therefore, the results of the experiments indicate that Prime's attach/detach behavior is likely not the culprit. Some notable observations from running these experiments (as well as other testing):

- No clear failure pattern
 - Can not appear for dozens of tests and then appear unexpectedly
- Occurs more frequently with higher number of emulated clients
- With most recent driver modifications, only happening with the final update

Theory:

- Driver is not stuck waiting to resend the packet (the WRITE_FD even listener addresses this)
- Driver THINKS it has successfully sent the final packet
 - Otherwise, the output would result in an inf loop of resend attempts instead of freezing
- Driver "thinks" it has successfully delivered the final packet → Prime receives the packet
 - Its not a delivery issue due to the socket attach/detach behavior
- For whatever reason, Prime fails to order/execute the final update and return a response to the driver
- This behavior was observed only with the default MAX_PO_IN_FLIGHT
- Next step: Trace Prime's handling of the final packet when this behavior occurs

MAX_PO_IN_FLIGHT Testing

Single Driver Configuration (250 Mbps link speed, 25k total sends, MAX_PO_IN_FLIGHT=20)						
		Throughput (updates/sec)		Latency (ms)		
Num. Emulated Clients	Elapsed Time (seconds)	Expected	Actual	Min	Max	Avg
25	24.15	1038.91	1035.24	17.08	45.318	24.064
50	24.21	1036.6	1032.76	24.572	77.56	48.234
75	24.36	1029.83	1026.37	16.938	99.882	72.827
100	24.43	1026.78	1023.39	24.385	137.508	97.392
200	24.3	1033.9	1028.7	20.202	220.201	193.443
Single Driver Configuration (250 Mbps link speed, 25k total sends, MAX_PO_IN_FLIGHT=75)						
		Throughput (updates/sec)		Latency (ms)		
Num. Emulated Clients	Elapsed Time (seconds)	Expected	Actual	Min	Max	Avg
25	20.88	1201.63	1197.21	17.48	43.516	20.805
50	19.64	1276.79	1273.01	19.469	55.103	39.161
75	13.4	1872.51	1865.03	19.204	50.224	40.053
100	13.26	1892.68	1885.39	20.578	92.562	52.835
200	13.25	1895.55	1886.82	27.421	131.899	105.511

